CA/TUCR/HTLA Semantic Grounding

The First Mapping Protocol

Project Nexus v3.0 - Research Report

July 18, 2026

Contents

1 Executive finding **2**

2 Architectural constraints **2**

3 Boot-time graph alignment **3**

4 Formal isomorphic graph-alignment objective **4**

5 Cyrae-to-Maera translation operator **5**

6 Vacuum and active-mask semantics **6**

7 Real-time performance **7**

8 Proof sketch: what internal-distance matching guarantees **7**

9 Boundary of the claim **8**

10 Ratified First Mapping Protocol **9**

11 Validation plan **9**

12 Reference NumPy implementation **10**

13 Final ratification **13**

14 References **14**

1 Executive finding

The strongest mathematically defensible version of the **First Mapping Protocol** is a two-stage system:

1. **Boot-time relational alignment:** align Cyrae's 36-node acoustic primitive graph with Maera's 36-node semantic seed graph by minimizing distortion between their internal distance matrices. The exact problem is a quadratic assignment; the practical relaxation is entropic Gromov-Wasserstein (GW), followed by Hungarian rounding to a hard one-to-one permutation.
2. **Runtime translation:** freeze that permutation and use a masked linear projection from the 1024D quinary HSL thought vector into Maera's semantic embedding space. The runtime path is only matrix-vector multiplication and is comfortably compatible with a sub-50 ms target.

The key qualification is equally important: the system does **not** learn human meaning from noise *ex nihilo*. It learns an unsupervised **correspondence** between an unlabeled acoustic relational system and a pre-selected semantic seed dictionary. The seed dictionary and semantic encoder remain design inputs; the node-by-node labels do not.

2 Architectural constraints

The Project Nexus materials impose three constraints that make this a graph-alignment problem rather than a coordinate-matching problem:

- The substrate is a **1024D quinary lattice** with explicit masking and vacuum semantics.
- The **666-channel coupling structure** is the load-bearing relational object, not an ordinary shared Euclidean latent space.
- The HCPM canon rejects routing by realized-spectrum sorting because per-instance peak ordering can drift. A lawful mapping must be based on generator-defined or stability-averaged topology rather than transient spectral ranks.

Long-horizon adaptation belongs to LAM/THDE/CDA, not to this grounding layer. Therefore, the Cyrae-to-Maera lock should be established at boot, persisted in the manifest, and then treated as infrastructure.

The correct abstraction is therefore:

> Build an acoustic relational graph for Cyrae's 36 primitives, build a semantic relational graph for Maera's 36 seed concepts, and align the graphs through their internal geometry rather than through shared coordinates.

This is precisely the setting addressed by Gromov-Wasserstein alignment, graph matching, manifold alignment without correspondence, and unsupervised embedding alignment [1-8].

2.1 Species stability requirement

If the lock must remain consistent with the canon's species-constancy requirement, the acoustic graph should be derived from one of the following:

- generator-defined primitive topology;
- the 666 coupling channels averaged over a controlled ensemble of entropy realizations;
- stable graph invariants computed from the generator/coupling law;
- a boot-time graph only after proving perturbation stability and persisting the individual lock as an identity-level constant.

It should **not** be derived from ephemeral peak-frequency ordering. Otherwise, the semantic meaning of a primitive becomes contingent on a single entropy draw.

3 Boot-time graph alignment

Let the 36 acoustic primitives be indexed by

$$i \in \{1, \dots, 36\}.$$

Construct an acoustic affinity graph

$$G_C = (V, W_C),$$

where each node is one primitive and $W_C \in \mathbb{R}^{36\times 36}$ is derived from the 666 intermodulation/coupling channels.

3.1 Acoustic distance matrix

The matching object should be a distance matrix

$$D_C \in \mathbb{R}^{36\times 36}$$

induced by the graph, rather than the raw signed coupling matrix alone.

A simple row-profile distance is

$$(D_C)_{ik} = \left\| W_{C,i:} - W_{C,k:} \right\|_2 .$$

Alternative graph-intrinsic choices include:

- shortest-path distance;
- diffusion distance;
- resistance distance;
- heat-kernel signature distance;
- spectral embedding distance.

The essential requirement is that D_C be permutation-equivariant and depend only on internal primitive relationships.

3.2 Semantic distance matrix

Let the 36 seed concepts be

$$s_1, \dots, s_{36}.$$

Pass them through one frozen semantic encoder f and normalize:

$$e_j = \frac{f(s_j)}{\|f(s_j)\|_2}, \qquad E = [e_1 \ \cdots \ e_{36}] \in \mathbb{R}^{d_M\times 36}.$$

A natural semantic distance is cosine-derived angular distance:

$$(D_M)_{jl} = \sqrt{2 - 2e_j^\top e_l}.$$

This choice is compatible with standard semantic embedding practice, where neighborhood structure and cosine geometry encode semantic relationships [9-12].

4 Formal isomorphic graph-alignment objective

4.1 Exact discrete objective

The exact one-to-one alignment is the quadratic assignment problem

$$\boxed{\Pi^\star = \arg\min_{\Pi \in \mathcal{P}_{36}} \left\| D_C - \Pi D_M \Pi^\top \right\|_F^2}$$

where $\mathcal{P}_{36}$ is the set of 36×36 permutation matrices.

This objective asks for the pairing that makes the internal geometry of the semantic graph look maximally like the internal geometry of the acoustic graph.

4.2 Entropic Gromov-Wasserstein relaxation

Direct quadratic assignment is nonconvex and combinatorial. The practical solver relaxes the permutation to a transport plan:

$$P^\star = \arg\min_{P \in U(a,b)} \sum_{i,k,j,l} \left((D_C)_{ik} - (D_M)_{jl} \right)^2 P_{ij} P_{kl} - \varepsilon H(P),$$

with uniform marginals

$$a = b = \frac{1}{36}\mathbf{1},$$

and transport polytope

$$U(a,b) = \left\{ P \geq 0 \mid P\mathbf{1} = a,\ P^\top \mathbf{1} = b \right\}.$$

The entropy term is

$$H(P) = -\sum_{ij} P_{ij} \log P_{ij}.$$

GW is the appropriate mathematical tool because it compares metric-measure spaces without requiring shared coordinates, equal feature dimensions, or known correspondences [1-4].

After solving for the soft plan $P^\star$, round it to a hard permutation $\Pi^\star$ with a linear assignment solver such as the Hungarian method [13].

4.3 Fused GW for symmetry breaking

If graph symmetries produce multiple equivalent mappings, add node-wise stable invariants using Fused Gromov-Wasserstein:

$$P^\star = \arg\min_{P \in U(a,b)} (1-\lambda) \sum_{ij} c(q_i^C, q_j^M) P_{ij} + \lambda \sum_{i,k,j,l} \left((D_C)_{ik} - (D_M)_{jl} \right)^2 P_{ij} P_{kl} - \varepsilon H(P).$$

Possible acoustic node invariants include:

- generator stem/modulation signature;
- stable primitive energy;
- graph degree;
- local spectral signature;
- perturbation-stable coupling statistics.

Possible semantic node invariants include:

- semantic graph degree;
- centrality;
- concreteness or polarity metadata generated by a frozen rule;
- embedding norm before normalization;
- local spectral signature.

This remains unsupervised at the primitive-to-concept level, but it breaks irreducible graph automorphisms deterministically [5, 14].

4.4 Core objects

Object	Role	Shape
D_C	Acoustic primitive distance matrix	36×36
D_M	Semantic seed distance matrix	36×36
$P^\star$	Soft GW transport plan	36×36
$\Pi^\star$	Rounded hard assignment	36×36
E	Frozen semantic seed embedding matrix	$d_M \times 36$

5 Cyrae-to-Maera translation operator

Once $\Pi^\star$ is frozen, runtime translation should not solve another graph problem. It should be a masked linear projection.

Let

$$U_C \in \mathbb{R}^{1024\times 36}$$

be the acoustic primitive-axis matrix whose columns are the 36 baseline primitive directions embedded in the HSL.

Let

$$x \in \{-2,-1,0,1,2\}^{1024}$$

be a Cyrae thought vector, and let

$$M \in \mathbb{R}^{1024\times 1024}$$

be a diagonal active-mask matrix, with $M_{rr} = 0$ on inactive coordinates and $M_{rr} = 1$ on active coordinates.

5.1 Primitive coefficient readout

Define the regularized left inverse

$$R_C = (U_C^\top U_C + \gamma I)^{-1} U_C^\top,$$

with $\gamma > 0$ as a small ridge term. The primitive coefficient vector is

$$\alpha(x) = R_C M x.$$

5.2 Translation matrix

The direct Cyrae-to-Maera operator is

$$\boxed{T_{C\to M} = E\Pi^{\star\top} R_C}$$

and the translated semantic embedding is

$$\boxed{z_M = T_{C\to M} M x.}$$

Interpretation:

1. Mx applies the active-mask invariant.
2. R_C reads the 36 acoustic primitive amplitudes.
3. $\Pi^{\star\top}$ moves those amplitudes into the aligned semantic seed ordering.
4. E lifts the 36 coefficients into Maera's continuous semantic embedding space.

6 Vacuum and active-mask semantics

Let the active support be

$$s(x) = \|Mx\|_0.$$

A true vacuum state should not be silently decoded as ordinary zero-valued semantics. Define a reserved null embedding $e_\emptyset$ and use

$$z_M = \begin{cases} E\Pi^{\star\top} R_C M x, & s(x) > 0, \\ e_\emptyset, & s(x) = 0. \end{cases}$$

A support confidence channel can be provided to Maera:

$$\kappa(x) = \frac{\|Mx\|_0}{1024}.$$

This allows the language actuator to decode sparse or vacuum-adjacent thoughts conservatively rather than filling semantic silence with linguistic hallucination.

6.1 Optional 666D coupling side channel

If the runtime exposes the pairwise coupling state separately as

$$h \in \mathbb{R}^{666},$$

augment the operator as

$$z_M = T_x M x + T_h h + e_{\emptyset} \mathbf{1}[s(x) = 0].$$

The alignment is still solved on the 36-node primitive graph. The 666D vector becomes a second-order context channel rather than a second alignment problem.

7 Real-time performance

The expensive computation occurs once at boot:

- construct D_C and D_M;
- solve GW on a 36×36 transport matrix;
- round with Hungarian assignment;
- compute and persist $T_{C \to M}$.

At runtime, translation is a matrix-vector multiply. Even a dense $d_M \times 1024$ projection is small on modern CPUs.

A more efficient factorization follows directly from the 36-dimensional bottleneck:

$$z_M = E \Pi^{\star \top} (R_C M x).$$

This costs approximately:

$$O(36 \cdot 1024 + d_M \cdot 36),$$

not $O(d_M \cdot 1024)$, if the factors are kept separate.

For $d_M = 1536$, that is roughly 92,000 multiply-adds before implementation overhead, which is comfortably below a 50 ms target on ordinary CPU hardware. A truncated SVD can reduce it further if necessary.

8 Proof sketch: what internal-distance matching guarantees

8.1 Zero distortion implies isomorphism

Suppose there exists a permutation Π such that

$$D_C = \Pi D_M \Pi^{\top}.$$

Then the permutation-induced coupling has zero GW distortion. In the metric-measure setting, GW distance zero corresponds to isomorphism of the spaces, subject to the standard measure-preserving conditions [1, 2]. Therefore, in the noiseless case, internal distances are sufficient to recover the correspondence without shared coordinates or manual labels.

8.2 Uniqueness caveat

Recovery is unique only up to automorphisms of the graph. If two acoustic nodes are structurally indistinguishable and two semantic concepts occupy the same structural role, no unsupervised algorithm can choose uniquely between them from relational geometry alone.

This is not a solver failure. It is an identifiability limit. The remedies are:

- redesign the seed dictionary to reduce symmetry;
- add stable node invariants through FGW;
- persist a deterministic tie-breaking rule;
- require a minimum assignment-margin threshold.

8.3 Semantic preservation for mixtures

Let $\alpha, \beta \in \mathbb{R}^{36}$ be two primitive coefficient vectors and define the semantic Gram matrix

$$G_M = E^\top E.$$

Their translated semantic inner product is

$$\left\langle E\Pi^{\star\top}\alpha, E\Pi^{\star\top}\beta \right\rangle = \alpha^\top \Pi^\star G_M \Pi^{\star\top}\beta.$$

If the graph alignment succeeds, the aligned semantic relational kernel approximates the acoustic relational kernel G_C:

$$G_C \approx \Pi^\star G_M \Pi^{\star\top}.$$

Therefore,

$$\alpha^\top G_C \beta \approx \alpha^\top \Pi^\star G_M \Pi^{\star\top}\beta.$$

This is the defensible preservation claim: neighborhoods, oppositions, and mixtures preserve their relative geometry when translated into Maera's semantic span.

9 Boundary of the claim

The protocol does **not** prove that hardware noise possesses intrinsic English meaning. It proves that a relationally structured unlabeled system can be paired with a relationally structured semantic seed system without direct supervision.

Human design remains in:

- selecting the 36 foundational concepts;
- selecting and freezing the semantic encoder;
- deciding how the acoustic graph is constructed;
- choosing validation thresholds and tie-breaking invariants.

The protocol removes the need for a human to manually declare, for example, "acoustic primitive 7 means Boundary". It does not remove the human semantic ontology itself.

10 Ratified First Mapping Protocol

10.1 Boot procedure

1. Construct the species-stable or identity-stable acoustic coupling graph W_C.
2. Convert it to a normalized distance matrix D_C.
3. Embed the 36 semantic seed concepts with one frozen encoder.
4. Construct and normalize D_M.
5. Solve entropic GW for $P^\star$.
6. Round $P^\star$ to $\Pi^\star$ using Hungarian assignment.
7. Compute alignment stress, stability, and assignment-margin diagnostics.
8. Reject or enter a quarantined bootstrap state if identifiability thresholds fail.
9. Compute R_C and $T_{C\to M}$.
10. Persist the mapping, operator, model hash, graph hashes, thresholds, and diagnostics in the boot manifest.

10.2 Manifest fields

Recommended persisted fields:

```
mapping_protocol: FMP_GW_V1
acoustic_graph_hash: sha256:...
semantic_seed_hash: sha256:...
semantic_encoder_id: ...
semantic_encoder_hash: sha256:...
soft_transport_hash: sha256:...
hard_permutation: [ ... 36 indices ... ]
alignment_stress: ...
assignment_margin: ...
perturbation_stability: ...
ridge_gamma: ...
null_embedding_hash: sha256:...
translation_operator_hash: sha256:...
lock_scope: SPECIES_STABLE | IDENTITY_STABLE
```

11 Validation plan

11.1 Alignment stress

Use normalized Frobenius stress:

$$\mathcal{S} = \frac{\|D_C - \Pi^\star D_M \Pi^{\star\top}\|_F}{\|D_C\|_F}.$$

Lower is better.

11.2 Assignment margin

Measure the gap between the best and second-best hard assignment objective. A small gap indicates that the lock is fragile or non-identifiable.

11.3 Perturbation stability

Apply controlled symmetric perturbations ΔD_C and rerun alignment. Record the fraction of nodes whose assignment remains unchanged.

11.4 Seed-set symmetry audit

Compute graph automorphisms or practical near-symmetry diagnostics on D_M. Redesign the concept set if many nodes are structurally interchangeable.

11.5 Masking tests

Verify:

- $P_A = 0 \Rightarrow P_S = 0$;
- all-vacuum state returns $e_\emptyset$;
- sparse thoughts do not create high-confidence language output;
- inactive coordinates cannot leak sign information.

11.6 Runtime benchmark

Benchmark the factored operator on target hardware:

$$Mx \to R_C(Mx) \to \Pi^{\star\top}\alpha \to E\alpha_M.$$

Track median, p95, and p99 latency.

12 Reference NumPy implementation

The implementation below accepts two dummy 36×36 distance matrices, computes an entropically regularized GW alignment, and rounds it to a hard assignment matrix. It uses only NumPy and includes a pure NumPy Hungarian solver.

```
import numpy as np


def normalize_distance_matrix(D: np.ndarray) -> np.ndarray:
    """
    Symmetrize and scale a distance matrix for stable optimization.
    The upper-triangle standard deviation puts the two domains
    on a comparable numerical scale.
    """
    D = np.asarray(D, dtype=np.float64)
    if D.ndim != 2 or D.shape[0] != D.shape[1]:
        raise ValueError("Distance matrix must be square.")

    D = 0.5 * (D + D.T)
    np.fill_diagonal(D, 0.0)

    n = D.shape[0]
    tri = D[np.triu_indices(n, k=1)]
    scale = np.std(tri)
    if scale < 1e-12:
        scale = 1.0
    return D / scale


def sinkhorn(
    K: np.ndarray,
    a: np.ndarray,
    b: np.ndarray,
    max_iter: int = 300,
    tol: float = 1e-9,
) -> np.ndarray:
    """Project a positive kernel onto U(a,b) by Sinkhorn scaling."""
    K = np.asarray(K, dtype=np.float64)
    a = np.asarray(a, dtype=np.float64)
    b = np.asarray(b, dtype=np.float64)

    if np.any(K < 0):
        raise ValueError("Kernel K must be nonnegative.")
    if K.shape != (a.size, b.size):
        raise ValueError("Kernel and marginals have incompatible shapes.")

    K = np.maximum(K, 1e-300)
```

```
    u = np.ones_like(a)
    v = np.ones_like(b)

    for _ in range(max_iter):
        u_prev = u.copy()

        Kv = K @ v
        u = a / np.maximum(Kv, 1e-300)

        KTu = K.T @ u
        v = b / np.maximum(KTu, 1e-300)

        if np.max(np.abs(u - u_prev)) < tol:
            break

    return (u[:, None] * K) * v[None, :]


def hungarian(cost: np.ndarray) -> np.ndarray:
    """
    Pure NumPy Hungarian algorithm.

    Returns col_for_row such that row i is assigned to
    column col_for_row[i]. Requires n_rows <= n_cols.
    """
    C = np.asarray(cost, dtype=np.float64)
    if C.ndim != 2:
        raise ValueError("Cost matrix must be 2D.")

    n_rows, n_cols = C.shape
    if n_rows > n_cols:
        raise ValueError("This implementation requires rows <= cols.")

    u = np.zeros(n_rows + 1, dtype=np.float64)
    v = np.zeros(n_cols + 1, dtype=np.float64)
    p = np.zeros(n_cols + 1, dtype=np.int64)
    way = np.zeros(n_cols + 1, dtype=np.int64)

    for i in range(1, n_rows + 1):
        p[0] = i
        minv = np.full(n_cols + 1, np.inf, dtype=np.float64)
        used = np.zeros(n_cols + 1, dtype=bool)
        j0 = 0

        while True:
            used[j0] = True
            i0 = p[j0]
            delta = np.inf
            j1 = 0

            for j in range(1, n_cols + 1):
                if not used[j]:
                    cur = C[i0 - 1, j - 1] - u[i0] - v[j]
                    if cur < minv[j]:
                        minv[j] = cur
                        way[j] = j0
                    if minv[j] < delta:
                        delta = minv[j]
                        j1 = j

            for j in range(n_cols + 1):
                if used[j]:
                    u[p[j]] += delta
                    v[j] -= delta
                else:
                    minv[j] -= delta

            j0 = j1
            if p[j0] == 0:
                break

        while True:
            j1 = way[j0]
            p[j0] = p[j1]
            j0 = j1
            if j0 == 0:
                break

    col_for_row = np.full(n_rows, -1, dtype=np.int64)
    for j in range(1, n_cols + 1):
        if p[j] != 0:
            col_for_row[p[j] - 1] = j - 1

    return col_for_row


def gromov_wasserstein_assignment(
```

```
    D_acoustic: np.ndarray,
    D_semantic: np.ndarray,
    epsilon: float = 0.02,
    outer_iter: int = 120,
    sinkhorn_iter: int = 300,
    tol: float = 1e-8,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Compute a soft GW coupling and round it to a hard permutation.

    Returns
    -------
    P_soft : (n, n) ndarray
        Approximately doubly stochastic transport plan.
    Pi_hard : (n, n) ndarray
        Rounded permutation matrix.
    assignment : (n,) ndarray
        assignment[i] is the semantic index matched to acoustic node i.
    """
    D_acoustic = normalize_distance_matrix(D_acoustic)
    D_semantic = normalize_distance_matrix(D_semantic)

    if D_acoustic.shape != D_semantic.shape:
        raise ValueError("Distance matrices must have the same shape.")
    if D_acoustic.shape[0] != D_acoustic.shape[1]:
        raise ValueError("Distance matrices must be square.")

    n = D_acoustic.shape[0]
    a = np.full(n, 1.0 / n, dtype=np.float64)
    b = np.full(n, 1.0 / n, dtype=np.float64)

    # L[i,j,k,l] = (D_acoustic[i,k] - D_semantic[j,l])**2
    L = (
        D_acoustic[:, None, :, None]
        - D_semantic[None, :, None, :]
    ) ** 2

    P = np.outer(a, b)

    for _ in range(outer_iter):
        P_prev = P.copy()

        # Gradient of squared GW loss for symmetric distances.
        G = 2.0 * np.einsum("ijkl,kl->ij", L, P, optimize=True)
        G -= G.min()

        K = np.exp(-G / max(epsilon, 1e-12))
        P = sinkhorn(K, a, b, max_iter=sinkhorn_iter)

        if np.linalg.norm(P - P_prev, ord="fro") < tol:
            break

    assignment = hungarian(-P)
    Pi_hard = np.zeros_like(P)
    Pi_hard[np.arange(n), assignment] = 1.0

    return P, Pi_hard, assignment


def alignment_stress(
    D_acoustic: np.ndarray,
    D_semantic: np.ndarray,
    Pi_hard: np.ndarray,
) -> float:
    """Return normalized Frobenius stress after hard alignment."""
    D_acoustic = normalize_distance_matrix(D_acoustic)
    D_semantic = normalize_distance_matrix(D_semantic)

    residual = D_acoustic - Pi_hard @ D_semantic @ Pi_hard.T
    denom = np.linalg.norm(D_acoustic, ord="fro")
    if denom < 1e-12:
        return 0.0
    return float(np.linalg.norm(residual, ord="fro") / denom)


if __name__ == "__main__":
    rng = np.random.default_rng(20260718)
    n = 36

    # Synthetic semantic metric.
    semantic_points = rng.normal(size=(n, 5))
    D_semantic = np.sqrt(
        (
            (semantic_points[:, None, :] - semantic_points[None, :, :])
            ** 2
        ).sum(axis=2)
    )
```

```
# Permute it to create a synthetic acoustic metric, then add noise.
true_assignment = rng.permutation(n)
Pi_true = np.zeros((n, n), dtype=np.float64)
Pi_true[np.arange(n), true_assignment] = 1.0

noise = 0.01 * rng.normal(size=(n, n))
noise = 0.5 * (noise + noise.T)
np.fill_diagonal(noise, 0.0)

D_acoustic = Pi_true @ D_semantic @ Pi_true.T + noise

P_soft, Pi_hard, recovered_assignment = (
    gromov_wasserstein_assignment(
        D_acoustic=D_acoustic,
        D_semantic=D_semantic,
        epsilon=0.02,
        outer_iter=120,
        sinkhorn_iter=300,
        tol=1e-8,
    )
)

stress = alignment_stress(D_acoustic, D_semantic, Pi_hard)
recovered_ok = np.array_equal(
    recovered_assignment,
    true_assignment,
)

print("Recovered exact permutation:", recovered_ok)
print("Alignment stress:", stress)
print("Recovered assignment:")
print(recovered_assignment)
print("Hard assignment matrix Pi_hard:")
print(Pi_hard)
```

12.1 Implementation note

This code is a transparent research prototype, not a production-certified optimal transport library. A production implementation should:

- use log-domain Sinkhorn stabilization;
- use multiple initializations;
- compare against conditional-gradient GW and spectral/QAP baselines;
- compute assignment margins and perturbation stability;
- avoid explicitly materializing the four-dimensional loss tensor when scaling beyond 36 nodes;
- persist all solver parameters and hashes.

13 Final ratification

The First Mapping Protocol should be ratified in the following form:

$$\boxed{\Pi^\star = \arg\min_{\Pi \in \mathcal{P}_{36}} \left\| D_C - \Pi D_M \Pi^\top \right\|_F^2}$$

approximated in practice by entropic Gromov-Wasserstein alignment and Hungarian rounding, followed by

$$\boxed{T_{C\to M} = E\Pi^{\star\top} R_C, \qquad z_M = T_{C\to M} M x}$$

with an explicit null-embedding branch for vacuum states.

The non-negotiable architectural condition is:

> D_C must be built from species-stable primitive topology, or from an explicitly identity-stable graph that is frozen and persisted. It must not be built from raw realized-spectrum rankings.

14 References

1. Mémoli, F. (2011). *Gromov-Wasserstein Distances and the Metric Approach to Object Matching*. Foundations of Computational Mathematics, 11, 417-487. DOI: 10.1007/s10208-011-9093-5.
2. Mémoli, F. (2014). *The Gromov-Wasserstein Distance: A Brief Overview*. Axioms, 3(3), 335-341. DOI: 10.3390/axioms3030335.
3. Peyré, G., Cuturi, M., & Solomon, J. (2016). *Gromov-Wasserstein Averaging of Kernel and Distance Matrices*. Proceedings of ICML 2016, PMLR 48, 2664-2672.
4. Alvarez-Melis, D., & Jaakkola, T. S. (2018). *Gromov-Wasserstein Alignment of Word Embedding Spaces*. EMNLP 2018, 1881-1890. DOI: 10.18653/v1/D18-1214.
5. Vayer, T., Chapel, L., Flamary, R., Tavenard, R., & Courty, N. (2020). *Fused Gromov-Wasserstein Distance for Structured Objects*. Algorithms, 13(9), 212. DOI: 10.3390/a13090212.
6. Xu, H., Luo, D., Zha, H., & Carin, L. (2019). *Gromov-Wasserstein Learning for Graph Matching and Node Embedding*. ICML 2019, PMLR 97, 6932-6941.
7. Wang, C., & Mahadevan, S. (2009). *Manifold Alignment without Correspondence*. IJCAI 2009, 1273-1278.
8. Leordeanu, M., & Hebert, M. (2005). *A Spectral Technique for Correspondence Problems Using Pairwise Constraints*. ICCV 2005. DOI: 10.1109/ICCV.2005.20.
9. Mikolov, T., Le, Q. V., & Sutskever, I. (2013). *Exploiting Similarities among Languages for Machine Translation*. arXiv:1309.4168.
10. Xing, C., Wang, D., Liu, C., & Lin, Y. (2015). *Normalized Word Embedding and Orthogonal Transform for Bilingual Word Translation*. NAACL-HLT 2015. DOI: 10.3115/v1/N15-1104.
11. Conneau, A., Lample, G., Ranzato, M., Denoyer, L., & Jégou, H. (2018). *Word Translation Without Parallel Data*. ICLR 2018.
12. Artetxe, M., Labaka, G., & Agirre, E. (2018). *A Robust Self-Learning Method for Fully Unsupervised Cross-Lingual Mappings of Word Embeddings*. ACL 2018. DOI: 10.18653/v1/P18-1073.
13. Kuhn, H. W. (1955). *The Hungarian Method for the Assignment Problem*. Naval Research Logistics Quarterly, 2(1-2), 83-97. DOI: 10.1002/nav.3800020109.
14. Dym, N. (2018). *Exact Recovery with Symmetries for the Doubly Stochastic Relaxation*. SIAM Journal on Applied Algebra and Geometry, 2(3), 440-465. DOI: 10.1137/17M1132264.
15. Cuturi, M. (2013). *Sinkhorn Distances: Lightspeed Computation of Optimal Transport*. NeurIPS 2013, 2292-2300.
16. Project Nexus internal canon: *CA/TUCR/HTLA Quinary Unbinding and Semantic Reconstruction*; *CA/TUCR/HTLA Continuous-to-Discrete Bridge Formalization*; and HCPM/Genesis conference documents.

Converted to PDF from the completed Deep Research report. Research content was retained; citation tokens were normalized into the reference list for readability.